CSCI1140 Tutorial 2

Hoover H.F. Yin

Department of Information Engineering
The Chinese University of Hong Kong

September 25, 2015

The story behind your code...

Contact

- YIN Ho Fai, Hoover
- Office: Institute of Network Coding (INC), 7/F, SHB
- Email: yhf015 at ie dot cuhk dot edu dot hk

- Your local tutor
- Can speak Cantonese :D

Outline

1. The story behind your code
    1. Why programming?
    2. C compiler
    3. Question everything you typed!
    4. Why data types?
    5. = vs ==
    6. Iterations

README

The first part may dig too deep for you. If you just want to finish your exercises, please only read the second part.

The first part is for you to understand everything you typed in your C code. It is for your own interest. Don't feel afraid if you can't understand the slides at this moment.

Notes: I use GCC to demonstrate the concepts

Historical Notes

- In the very old days, the full name of GCC is GNU C Compiler
- Nowadays, the full name of GCC is GNU Compiler Collection
- What is GNU? Search "GNU Project"

Why Programming?

Computer is stupid, but loyal

- Stupid means:
  - It can't think
  - It can only do simple operations (e.g. +-*/)
  - It can only follow instructions sequentially
- Loyal means:
  - It won't question your instructions
  - It follows your instructions exactly
  - If there's a bug, it must be your problem

You need to write instructions on how to solve your problem!
How? Programming

Why Programming?

- Computers can't understand your C code!
- Why do we write in C !!!!????
  - Human readable
  - Better view to program flows
  - Can perform low-level jobs like controlling the memory
- What can a computer understand? Machine Language
- How to let a computer understand your C code? Compile it!

What machine language looks like?

01001010001010010010110101001010 10

I think you don't want to code in this way...

C Compiler

Contains 4 major components:

1. Preprocessor
   - Process `#include`, `#define`, etc.
   - Output a processed source file
2. Compiler
   - Turn the processed code into assembly code
   - (optional) Optimize your code
3. Assembler
   - Translate the assembly code into machine code
   - The output is called object file
4. Linker
   - Combine object files into an executable

Preprocessor

Basically every line starts with # is for the preprocessor

Example of `#include`:

first.c

```
#include "first.h"

int main(void)
{
    //Hello :D
    int hi = 0;
    return 0;
}
```

first.h

```
int i;
float j;
double k;
```

"xxx" means it's a custom file in your working directory, while <xxx> means it's a file in the default directory.

Try to find out where is `stdio.h` in your computer!

Preprocessor

Compilation Command

```
gcc -E first.c -o first.i
```

- `-E` means only run the preprocessor
- `-o first.i` means write the output in a file named `first.i`
- `.i` is the extension for preprocessed files
- The full command means: Run preprocessor on the file named `first.c` and output the result in another file named `first.i`

#include

The output of the preprocessor:

first.i

```
# 1 "first.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "first.c"
# 1 "first.h" 1
int i;
float j;
double k;
# 2 "first.c" 2
```

first.i (cont.)

```
int main(void)
{

 int hi = 0;
 return 0;
}
```

#include

- The lines start with # in `first.i` are comments
- The preprocessor copy and paste the `#include` files!
- Comments in C code are deleted, but not necessary
- Tabs are substituted into spaces, but not necessary

Historical Notes

Old style preprocessor won't do the "not necessary" part. Try the command

```
gcc -E first.c -o first2.i -traditional
```

printf and stdio.h

Why `#include <stdio.h>` if we use `printf`?

helloworld.c

```
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

Compilation Command

```
gcc -E helloworld.c -o helloworld.i
```

printf and stdio.h

helloworld.i is very long... What we want is the line

helloworld.i (partial)

```
extern int printf (__const char *__restrict __format,
                   ...);
```

What we want is the function prototype of printf!

printf and stdio.h

The prototype seems complicated...

- `extern`? It can be omitted. It means the function is not in this source file.
- `__const`? Just think it as `const`.
- `__restrict`? Search "restrict pointer" if you know pointers.
- `...` in the prototype? Search "`stdarg.h`".
- `printf returns int`? Yes, `printf` returns the number of characters printed on the screen, or a negative number if there's any error.
- What will happen if we don't have the function prototype? We will discuss it later.

#define

What is #define? Search and replace!

define.c

```
#include <stdio.h>

#define f(x) x*x

int main(void)
{
    int a = f(5);
    int b = f(2+3);
    printf("%d, %d\n", a, b);
    return 0;
}
```

Guess the output∼

#define

Compilation Command

```
gcc define.c -o define
```

- It means compile `define.c` into an executable named `define`
- * Please name it `define.exe` if you are using Windows
  - I won't mention it in the later slides
- `.exe` means executable. It is not necessary in Unix, Linux, Mac OS.

Execution Command

```
./define
```

Output

```
25, 11
```

#define

Why a $\neq$ b? Because it's search and replace!

Compilation Command

```
gcc -E define.c -o define.i
```

define.i (partial)

```
int a = 5*5;
int b = 2+3*2+3;
```

Therefore we usually use a lot of parentheses in #define, e.g.

```
#define f(x) ((x)*(x))
```

Compiler

Compiler?

- Turn the output of the preprocessor into assembly code
- (optional) Optimize your code

Assembly Language?

- Basically every line corresponds to one machine code
- Machine dependent!
  - Different between CPU architectures, like Intel and ARM
  - Different styles, like Intel syntax and GAS syntax
  - You will learn more in ELEG3230

Assembly Language

Why don't we write in assembly language? Consider the following code.

if.c

```
#include <stdio.h>

int main(void)
{
    int a;
    if(a > 0)
        puts(":)");
    else
        puts(":(");
    return 0;
}
```

Assembly Language

- There's no error if you don't initialize a variable, but you don't know what was left in the memory before you run your program!
- The full name of `puts` is put string. It will print a string on the screen and then open a new line
  - The results of `puts(":)");` and `printf(":)\n");` are the same

Compilation Command

```
gcc -S if.c -o if.s
```

- `-S` means output the assembly code
  - pass all `.c` files through preprocessor and compiler
  - pass all `.i` files through compiler only
- `.s` is the extension for assembly codes

Assembly Language

The assembly file is in GAS syntax. We are not going to read the assembly code.
The following section corresponds to our `if-else` block.

`if.s` (partial)

```
    cmpl    $0, -4(%rbp)
    jle     .L2
    movl    $.LC0, %edi
    call    puts
    jmp     .L3
.L2:
    movl    $.LC1, %edi
    call    puts
.L3:
```

Assembly Language

If we write it in C in a similar way...

```
    if(a <= 0)
        goto L2;
    puts(":)");
    goto L3;
L2:
    puts(":(");
L3:;
```

Why don't we write in assembly language?

- It is not easy to see the program flow
  - Think what will happen if we have nested-`if`...
- Clumsy. One simple code in C may results in multiple assembly codes

Optimization

We can ask the compiler to optimize our code.

opt.c

```
#include <stdio.h>

int main(void)
{
    int a = 2;
    a += 2;
    a += 2;
    a += 2;
    a += 2;
    printf("%d\n", a);
    return 0;
}
```

Optimization

We compare the assembly codes...

Compilation Command

```
gcc -O0 -S opt.c -o opt0.s
gcc -O1 -S opt.c -o opt1.s
```

- -O0 means no optimization
- -O1 means level 1 optimization
- we also have -O2 and -O3 for further performance optimization
- and we have -Os for size optimization

Optimization

With -O0, our C code is translated directly...

opt0.s (partial)

```
    movl    $2, -4(%rbp)
    addl    $2, -4(%rbp)
    addl    $2, -4(%rbp)
    addl    $2, -4(%rbp)
    addl    $2, -4(%rbp)
```

opt.c (partial)

```
    int a = 2;
    a += 2;
    a += 2;
    a += 2;
    a += 2;
```

With -O1...

opt1.s (partial)

```
    movl    $10, %esi
```

It simply means `int a = 10;`

Although computer calculates very fast, but remember, every instruction takes time! You will know more in CSCI2100.

Assembler

Assembler translates the assembly code into machine code.

main.c

```
#include <stdio.h>

//extern float foo(int);

int main(void)
{
    float b = foo(2.71828);
    printf("%f\n", b);
    return 0;
}
```

foo.c

```
#include <stdio.h>

float foo(int a)
{
    printf("%d\n", a);
    return 3.14159;
}
```

Assembler

Compilation Command

```
gcc -c main.c -o main.o
gcc -c foo.c -o foo.o
```

- `-c` means output the object file
  - similarly, we can pass in `.i` and `.s` files to skip the corresponding components
- `.o` is the extension for object files
- We can create the object files even
  - The function `foo()` is not in `main.c`
  - No `main()` in `foo.c`

Linker

Linker is the final component. It groups object files together and output an executable.

Compilation Command

```
gcc main.o foo.o -o main
```

- Note we do not have the function prototype of `foo()` in `main.c` but we successfully compile the codes
- So what will happen if we don't have the function prototype?

Execution Command

```
./main
```

Output

```
1
1078530048.000000
```

Function Prototype

The output is not as expected... If we uncomment the line

```
//extern float foo(int);
```

and compile the files, we will get

Output

```
2
3.141590
```

With the prototype

- the parameter `2.71828` is changed into `int` first
- the return value is a `float`

Without the prototype

- we pass `2.71828` and the function `foo()` treats it as an `int`
- by default, the return value is assumed to be `int`

Hello World!

Let's Hello World!

```
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

- Why called `stdio.h` and `printf`?
- Why return 0?

Historical Notes

The design of C was based on another language called B.
C was named because it's "after B".

Hello World!

`stdio.h`??

- The full name of `stdio` is Standard I/O (I/O means input/output)
- `.h` is the extension of header file, which should be `#include` in the header of your code
- Why the prototype of `printf` is in `stdio.h`?
  - Because it's a standard output function!

`printf`??

- The full name of `printf` is Print Formatted
- The first parameter is called a formatted string
- The output follows the format you specified in the formatted string

Hello World!

Return value in `main()`?

- Why `main()` is executed? There must be something called our `main()`!
- The return value will finally pass to the process which call our program

return0.c

```
int main(void)
{
    return 0;
}
```

return1.c

```
int main(void)
{
    return 1;
}
```

Let's get the return value!

Hello World!

Compilation Command

```
gcc return0.c -o return0
gcc return1.c -o return1
```

To get the return value, different shells have different way... Here's some examples

bash

```
echo $?
```

csh

```
echo $status
```

cmd (Windows)

```
echo %ERRORLEVEL%
```

- `echo` is self-explanatory: repeat what you said
- The shell actually substitute the 'variable' part before calling `echo`

Hello World!

Change the echo line according to your shell.

Execution Command

```
./return0
echo $?
```

Output

```
0
```

Execution Command

```
./return1
echo $?
```

Output

```
1
```

We did it! Then, what does 0 means?

- Exit successfully (no error)

Oh our program exited successfully... then what?

Hello World!

Useful when you run programs one by one...

Execution Command

```
./return0 && ./define
```

Output

```
25, 11
```

Execution Command

```
./return1 && ./define
```

Output (Nothing)

The command means:

- First execute `./return0` (or `./return1`)
- If it returns 0, then run `./define`
  - So `./define` is not executed after `./return1`

The && is similar to that in C (logical AND). Beware:

- 0 means false in C
- 0 means true in shells
  - There can be many kinds of errors, but (mostly) one kind of success

Comments

- Two types: // and /* */

comment.c

```
#include <stdio.h>

main()
{
    int c = 6 //*comment*/ 3
    ;;;;;;;;;;
    printf("%d\n", c);
}
```

- Is the code correct?
- What is the output?

Comments

There is no ; in this line... Is that okay?

```
int c = 6 //*comment*/ 3
```

- No one said the ; must in the same line!
- The first ; in this next line ends this statement

How about this line... There are some many ;s! Is that okay?

```
;;;;;;;;;;
```

- Yes! Each ; means the end of a statement
- Statement with nothing is called an empty statement
- It just do nothing :D

Comments

We wrote `main()` but not `int main(void)`... What will happen?

- The compiler won't check your code!

check.c

```
#include <stdio.h>

foo()
{
    printf("foo\n");
}

main()
{
    int a = foo(1);
}
```

Comments

The prototype is `foo()` but we use it as

```
int a = foo(1);
```

We can still compile this code without error!

Compilation Command

```
gcc check.c -o check
```

- Computer just follows your instructions!
- You should know what you are doing!

If you change the line `foo()` into

```
void foo(void)
```

Compilation error!!! Why?

Comments

It's because...

- Compiler now checks how you use the function `foo()`
- `void` means nothing, so compiler blames you if you have something~

For `void foo(void)`

- You should give no arguments to the function
- The function has no return value so you should not get something from it

Then... what does `main()` return to the shell?

Comments

Let's check it!

Execution Command

```
./check
echo $?
```

Output

```
foo
4
```

In fact it's an undefined behaviour. The 4 here is in fact the return value of `printf`. Why? It's about how the computer calls functions! Take CSCI3150/CENG3150 and you will know why!

Comments

Now, what is the output? Which kind of comment is deleted first?

- If `//` first, then $c = 6$ so the output is 6
- If `/* */` first, then $c = 6/3$ so the output is 2

Let's verify it!

Compilation Command

```
gcc comment.c -o comment
```

Execution Command

```
./comment
```

Output

```
6
```

Conclusion: `//` is removed first!

Comments

Historical Notes

Track back the history of C standard, the comment style // was first standardized in C99 (in 1999)! But this style is too useful, many compilers before 1999 also supported //. Try to compile in the very old school ANSI C standatd (which does NOT support //) with the option `-ansi`.

```
gcc comment.c -o comment -ansi
```

You will get the output 2 but not 6!

Data Types

How does the computer store a number?

$$01010100101010001010010100111010$$

Each binary digit uses a bit. A few bits can only represent a small number of combinations.

- Therefore we used to use a group of bits: 8 bits = 1 byte

$$01010100\ 10101000\ 10100101\ 00111010$$

- A 4 bytes unsigned integer can represent $[0, 2^{32} - 1] = [0, 4294967295]$

What will happen if we add 1 to 4294967295?

- Overflow!
- It becomes 0 as the it cannot store the most significant bit

Data Types

Difference between `float` and `double`?

- `float` uses 4 bytes, `double` uses 8 bytes
- So `double` can store a more accurate number

Why called `float`?

- Floating point number
- The idea is, we first store an integer, e.g. 314159265
- Then we store where to put the decimal point
- The point is floating, we can move it to other places
- A mathematical representation: scientific notation

Why called `double`?

- Double precision

Data Types

Let see how many digits `float` and `double` can store!

float.c

```
#include <stdio.h>

#define NUM .777777777777777777
int main(void)
{
    float f = NUM;
    double d = NUM;
    printf("      12345678901234567890\n");
    printf("f = %.60f\n"
           "d = %.60f\n", f, d);
    return 0;
}
```

Data Types

Wait, the string in `printf` is in 2 lines?

- Yes! It will concatenate automatically

Compilation Command

```
gcc float.c -o float
```

Execution Command

```
./float
```

Output

```
      12345678901234567890
f = 0.777777791023254394531250000000000000000000000000000000000000000
d = 0.777777777777777790113589162501739338040351867675781250000000
```

`float` promises 6 digits and `double` promises 15 digits (Can you see why they are not 7 and 16? Hint: Rounding off)

Data Types

What do `%d`, `%f`, `%c` mean in `printf`?

- decimal, not double!
- float
- character

= vs ==

equal.c

```
#include <stdio.h>

int main(void)
{
    int a = 10;
    if(a = 1)
        puts("A");
    else
        puts("B");
    printf("%d\n", a);
    return 0;
}
```

equal2.c

```
#include <stdio.h>

int main(void)
{
    int a = 10;
    if(a == 1)
        puts("A");
    else
        puts("B");
    printf("%d\n", a);
    return 0;
}
```

- Why no syntax error?

= vs ==

In C, 0 means false and non-0 means true!

- `if(a = 1)` means
  1. Assign 1 to a
  2. Ask if a is true or not
- `if(a == 1)` means
  1. Check if a equals 1. If yes, `a == 1` becomes 1, otherwise 0.
  2. Ask the value is true or not

So `a = 1` always true!

Compilation Command

```
gcc equal.c -o equal
gcc equal2.c -o equal2
```

Can you guess the output now?

= vs ==

Execution Command

```
./equal
```

Output

```
A
1
```

Execution Command

```
./equal2
```

Output

```
B
10
```

Loops

- What goto think

```
int i = 0;
start:
if(i >= 10)
    goto end;
printf("%d\n", i);
i++;
goto start;
end:;
```

- What while think

```
int i = 0;
while(i < 10){
    printf("%d\n", i);
    i++;
}
```

- What for think

```
int i;
for(i = 0; i < 10; i++)
    printf("%d\n", i);
```

- What you actually think

```
Huh?
```

Loops

The output of the `for` and `while` loops are the same

- And in fact they will change to the `goto` version, which is computer understandable

What is `goto`?

- Jump to the flag directly!
- The lines `start:` and `end:` are the ways to declare flags
- A semicolon after `end:` to prevent no statement after that

Let's try it!

Loops

goto.c

```
#include <stdio.h>

main()
{
    int i = 0;
start:
    if(i >= 10)
        goto end;
    printf("%d\n", i);
    i++;
    goto start;
end:;
}
```

Loops

Compilation Command

```
gcc goto.c -o goto
```

Execution Command

```
./goto
```

Output

```
0
1
2
3
4
5
6
7
8
9
```

for loops

Any syntax error?

```
for(;;);
```

- We can keep the statements in `for` empty
- Empty condition means always true!
- The last semicolon means the loop body is an empty statement
- So the loop is equivalent to `while(1){}`

```
for(int i = 0; i; i++);
```

- Variable declaration inside? It is valid after C99 standard
- Note the scope of the variable is the loop only
- The condition statement is `i` only? Yes, 0 means false while non-0 means true

for loops

So what is the scope of i if we declare outside?

for.c

```
#include <stdio.h>

int main(void)
{
    int i = -1;
    for(i = 0; i < 10; i++)
        printf("%d\n", i);
    puts("Loop ended");
    printf("%d\n", i);
    return 0;
}
```

for loops

compilation command

```
gcc for.c -o for
```

execution command

```
./for
```

output

```
0
1
2
3
4
5
6
7
8
9
loop ended
10
```

for loops

Infinite loop?

```
unsigned i = 1;
for(; i; i++);
```

- i will overflow and become 0
- Not an infinite loop

```
float f = .1;
for(; f != .1; );
```

- f not equals .1
- Why? Because by default .1 is a double
- f is casted into double, and the value is not equals to the double .1
- It's an infinite loop!

for loops

So how to test if f equals the float .1?

- .1f means a float .1

forone.c

```
int main(void)
{
    float f = .1;
    for(; f != .1f; );
    return 0;
}
```

Compilation Command

```
gcc forone.c -o forone
```

Execution Command

```
./forone
```

This time the loop will end.

for loops

What is the output?

one.c

```
#include <stdio.h>

int main(void)
{
    int i;
    double d = 0;
    for(i = 0; i < 10; i++)
        d += .1;
    printf("%d\n", d == 1.);
    return 0;
}
```

for loops

As the computer stores binary number, so it can only store a number in this form

$$\sum_{i \in \mathbb{Z}} a_i 2^i$$

where a_i are either 1 or 0. Can you express 0.1 in the format $a_{-1}\frac{1}{2} + a_{-2}\frac{1}{4} + a_{-3}\frac{1}{8} + \ldots$ with a limited number of terms?

- Sorry, no.
- Therefore the computer can't store the exact value of 0.1
- Sum up to 10 times can only get a number close to 1

for loops

compilation command

```
gcc one.c -o one
```

execution command

```
./one
```

output

```
0
```

of elements in an array

If I want to print all the elements in an array, what should I fill in?

```
int a[] = {1,2,3,4,5,6,7};
int i;
for(i = 0; i < /* fill in your code */; i++)
    printf("%d\n", a[i]);
```

- `sizeof(a)` will get the size of the whole array!
- To get the number of elements, we need to divide the size of each element
- `sizeof(a)/sizeof(int)`

of elements in an array

extra.c

```
#include <stdio.h>

int main(void)
{
    int a[] = {1,2,3,4,5,6,7};
    int i;
    for(i = 0; i < sizeof(a)/sizeof(int); i++)
        printf("%d\n", a[i]);
    return 0;
}
```

Compilation Command

```
gcc extra.c -o extra
```

of elements in an array

Execution Command

```
./extra
```

Output

```
1
2
3
4
5
6
7
```